ЛАБОРАТОРНАЯ РАБОТА № 3

Backend с REST API и базой данных

Проект «АвтоДеталь» · Каталог автозапчастей

Отчёт о выполнении · 30 октября 2026 года

Цель работы

Разделить существующее веб-приложение на клиентскую и серверную части; реализовать REST API на Python, хранение данных в реляционной БД, автоматическую документацию OpenAPI и взаимодействие frontend с backend через fetch.

Исходное состояние

В ЛР2 каталог был реализован на HTML, CSS и JavaScript. Исходные шесть деталей находились в массиве defaults, изменения сохранялись в localStorage браузера. Были доступны поиск, фильтрация, сортировка, добавление и удаление карточек.

Полученный результат

Сохранены оформление и функции ЛР2. Постоянные данные перенесены в SQLite. Добавлены сущности Part и Category, полный CRUD запчастей, редактирование через интерфейс, серверная валидация, обработка ошибок и Swagger UI. Frontend загружает каталог и справочник категорий через HTTP.

Архитектура и технологии

Компонент	Реализация
Клиент	HTML5, CSS3, Vanilla JavaScript; временный state для отображения
HTTP API	Python, FastAPI, Pydantic; GET / POST / PATCH / DELETE
Доступ к данным	SQLAlchemy, отдельная сессия на запрос, транзакции и rollback
Хранение	SQLite, файл data/autodetal.db, две связанные таблицы
Документация	OpenAPI 3.1, локальный Swagger UI 5.34.1

Поток данных: браузер отправляет fetch → FastAPI проверяет запрос → SQLAlchemy обращается к SQLite → сервер возвращает JSON → клиент обновляет карточки.

1. Сущности и структура БД

Основная сущность — запчасть Part. Связанная сущность — категория Category. Отношение 1:N: одной категории соответствует несколько запчастей. Связь задаётся внешним ключом parts.category_id → categories.id.

Поле Part	Тип в модели	Назначение / ограничение
id	INTEGER	PRIMARY KEY; генерируется автоматически
name	VARCHAR(60)	NOT NULL; 3–60 символов
description	TEXT	NOT NULL; 10–240 символов
article	VARCHAR(20)	NOT NULL, UNIQUE; 4–20 символов, верхний регистр
price	NUMERIC(10,2)	NOT NULL; 1…1 000 000 ₽, до 2 знаков после запятой
category_id	INTEGER	NOT NULL, FOREIGN KEY; индекс
parameter	VARCHAR(120)	Характеристика; значение по умолчанию
image_url	VARCHAR(100)	Одна из 6 локальных SVG-иллюстраций
created_at	DATETIME	NOT NULL; время создания, API возвращает UTC

Таблица categories

id — целочисленный первичный ключ; name — уникальное обязательное название; slug — уникальный обязательный код. Созданы категории: Двигатель, Тормоза, Зажигание, Подвеска, Привод и Другое.

Ограничения целостности

NOT NULL запрещает отсутствие обязательных значений. UNIQUE исключает одинаковые артикулы и дубликаты категорий. CHECK ограничивает цену, дробные копейки, длины строк и перечень изображений. FOREIGN KEY запрещает ссылку на отсутствующую категорию; ON DELETE RESTRICT запрещает удаление используемой категории.

В SQLite внешние ключи включаются через PRAGMA foreign_keys=ON для каждого нового соединения. Значения передаются в SQL через ORM, без склеивания строк запросов.

Первичное заполнение и сохранение

При первом запуске create_all создаёт таблицы, затем добавляются шесть категорий и шесть исходных запчастей ЛР2. Заполнение выполняется, только пока нет категорий. Поэтому удалённые детали не восстанавливаются при новом запуске, даже если пользователь удалил весь каталог.

В готовой демонстрационной БД семь деталей: шесть исходных и салонный фильтр, созданный через POST при проверке. Полная фактическая DDL-схема сохранена в docs/schema.sql.

Цена проходит проверку как Decimal; JSON возвращает строку с двумя десятичными знаками. SQLite использует NUMERIC affinity; проект не выполняет расчётов денежных проводок.

2. Контракт REST API

Метод	Путь	Назначение	Статус
GET	/api/parts	Список деталей	200
POST	/api/parts	Создание	201
GET	/api/parts/{part_id}	Чтение по ID	200
PATCH	/api/parts/{part_id}	Изменение полей	200
DELETE	/api/parts/{part_id}	Удаление	204
GET	/api/categories	Справочник категорий	200
GET	/api/categories/{category_id}	Категория по ID	200
GET	/api/health	Проверка сервера и БД	200

Создание запчасти

```
POST /api/parts
Content-Type: application/json

{
  "name": "Салонный фильтр",
  "description": "Очищает воздух в салоне автомобиля от пыли.",
  "article": "AP-CB-008",
  "price": "790.50",
  "category_id": 1,
  "parameter": "Угольный, 200 × 180 мм",
  "image_url": "/assets/images/filter.svg"
}
```

Ответ 201 Created содержит созданный объект, id, время создания и вложенную категорию. Заголовок Location указывает адрес /api/parts/ID. Артикул перед сохранением очищается от крайних пробелов и приводится к верхнему регистру.

Обновление и удаление

```
PATCH /api/parts/ID
{"price": "849.90"}

DELETE /api/parts/ID
```

PATCH изменяет только переданные поля; отсутствующие поля остаются прежними. Пустой PATCH возвращает 400, явный null и неизвестные поля — 422. DELETE возвращает 204 без тела ответа. Повторное чтение удалённого ID возвращает 404.

Обработка ошибок

400 — повторный артикул, неизвестная категория, пустой PATCH или повреждённый JSON. 404 — запись не найдена. 422 — неправильный тип, длина, диапазон или обязательное поле. 500 — ошибка сервера; внутренний SQL и настройки не раскрываются. После ошибки ограничения БД транзакция откатывается.

3. Реализация и запуск

Последовательность выполнения

1. Проанализированы массив defaults, localStorage и обработчики событий ЛР2; выделены запчасть и категория.

2. Созданы SQLAlchemy-модели, внешние ключи, ограничения и независимые от рабочей папки настройки подключения.

3. Добавлены Pydantic-схемы создания, частичного изменения и ответа. Реализованы функции CRUD и маршруты API.

4. Удалены локальный массив и сохранение каталога в localStorage. Начальная загрузка выполняет GET /api/categories и GET /api/parts.

5. Форма добавления отправляет POST, форма редактирования — PATCH, кнопка удаления — DELETE. Для редактирования сначала читается актуальная запись через GET по ID.

6. Добавлены состояния загрузки и ошибки, защита от повторной отправки формы, повторная загрузка, подтверждение удаления и управление фокусом модального окна.

7. Подключены Swagger UI и OpenAPI; проведены интеграционные и браузерные проверки.

Ключевой фрагмент взаимодействия

```
const [categories, items] = await Promise.all([
  api("/api/categories"),
  api("/api/parts")
]);
state.categories = categories;
state.items = items;
render();
```

Функция api использует fetch, проверяет response.ok, разбирает JSON и ошибки, возвращает null для 204 и прерывает запрос через 10 секунд. Карточки создаются DOM-методами; пользовательский текст вставляется через textContent.

Запуск на Windows

Распаковать архив, установить Python 3.10+ и запустить start.bat. При первом запуске устанавливаются зависимости и создаётся виртуальное окружение. Окно сервера необходимо оставить открытым.

```
Сайт:    http://127.0.0.1:8000
Swagger: http://127.0.0.1:8000/docs
OpenAPI: http://127.0.0.1:8000/openapi.json
```

Backend одновременно раздаёт frontend, поэтому второй сервер не требуется. Альтернативный запуск через порт 5500 описан в README; для него настроен ограниченный CORS. SQLite запускается вместе с backend и сохраняется в файле data/autodetal.db.

4. Frontend с данными сервера

Каталог после GET /api/parts. Сохранены фильтры, сортировка и иллюстрации ЛР2; карточки содержат ID из БД, цену и кнопки CRUD. В каталоге семь записей.

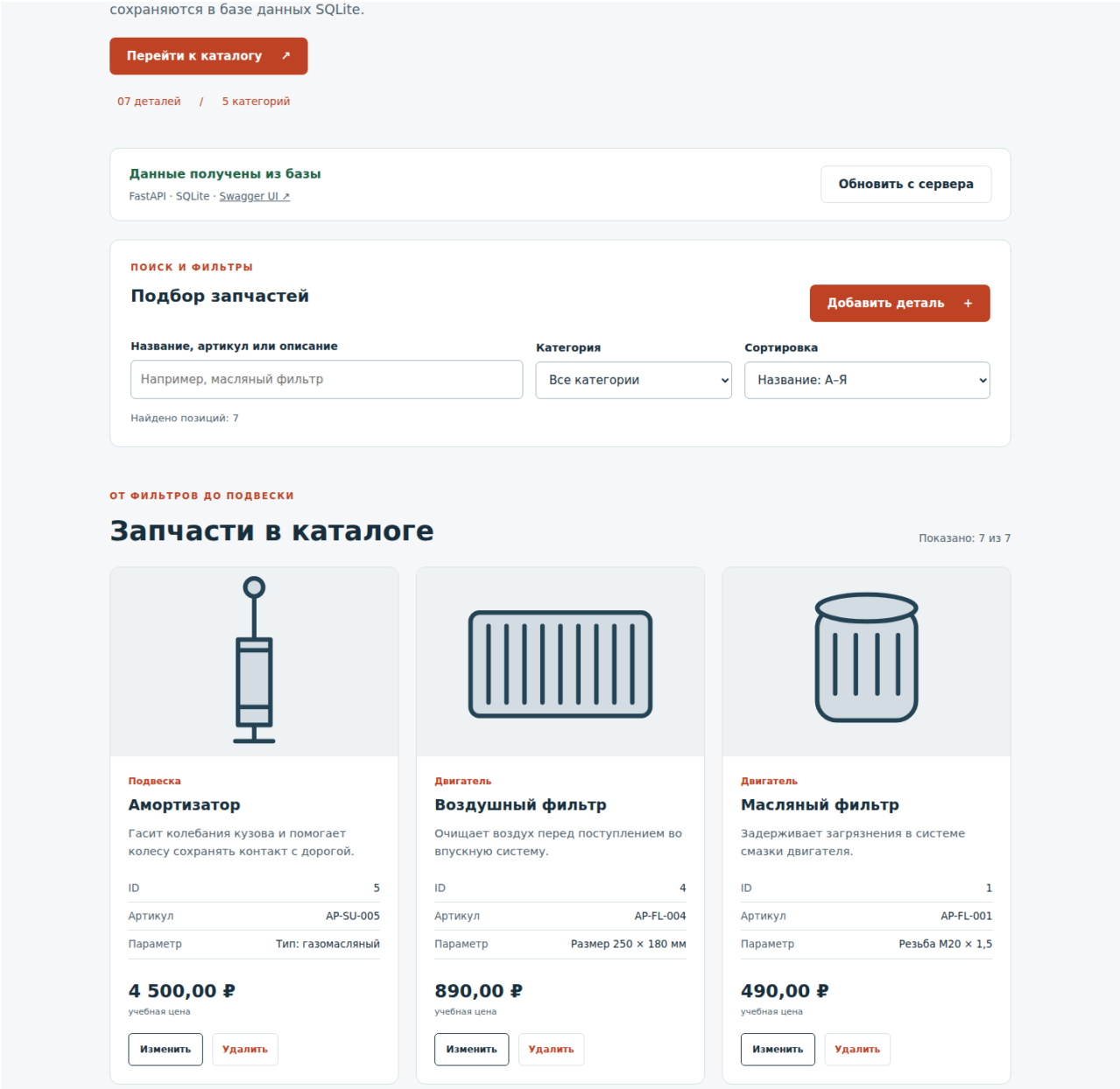


Рисунок 1 — Работающий интерфейс после загрузки данных из REST API

Проверены создание, изменение цены и категории, удаление, перезагрузка страницы, перезапуск backend и отсутствие горизонтальной прокрутки на экране 390 px. Дополнительный снимок телефона: docs/screenshots/04-mobile.png.

5. Автоматическая документация

FastAPI генерирует OpenAPI на основе маршрутов и Pydantic-схем. Swagger UI отображает методы, параметры пути, тела запросов, схемы ответов, статусы и примеры. Локальные JS/CSS включены в проект, доступ к CDN для /docs не нужен.

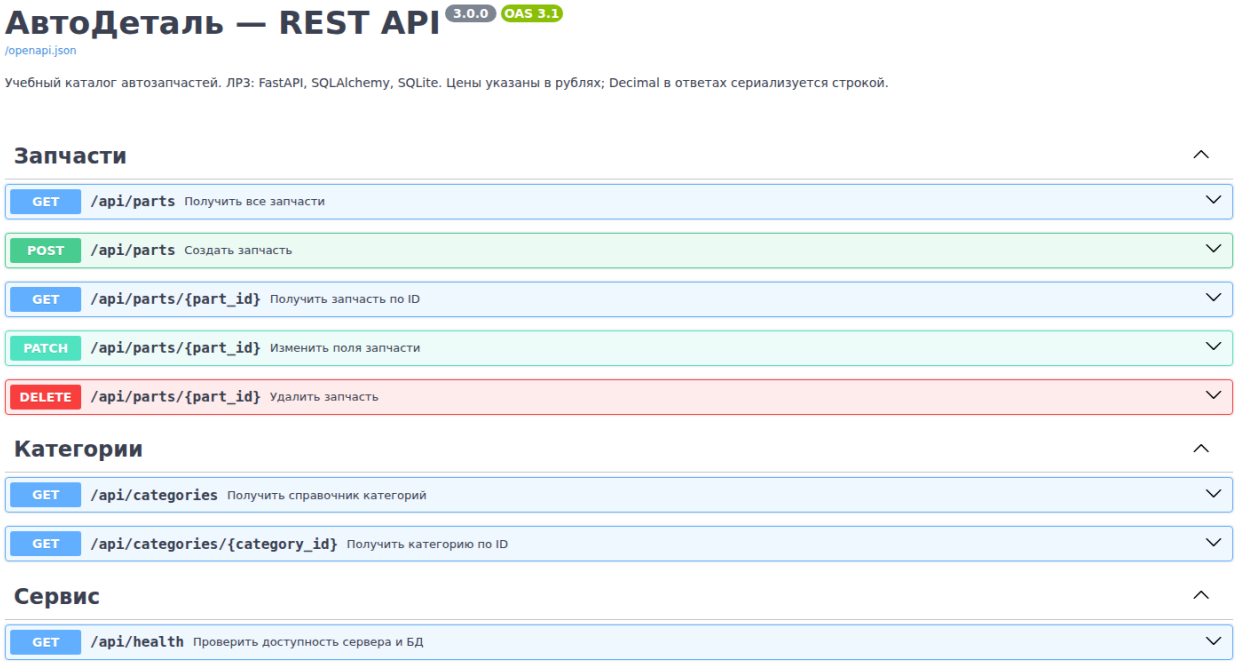


Рисунок 2 — Swagger UI: все реализованные операции API

Выполнение запроса из Swagger

Раскрыть GET /api/parts, нажать Try it out → Execute. Сервер возвращает 200 и JSON со списком деталей. Фактический снимок раскрытого метода и исполненного запроса сохранён в docs/screenshots/02-swagger.png.

Для POST вводится JSON с новым артикулом; ответ 201 содержит id. Затем по этому id выполняются GET, PATCH и DELETE. Спецификация доступна в /openapi.json; её выгруженная копия включена в docs/openapi.json.

6. Проверка хранения в SQLite

Для независимого просмотра данных выполнен SELECT через модуль sqlite3 стандартной библиотеки. Скрипт scripts/inspect_db.py открывает файл в режиме read-only, выводит строки и создаёт HTML-представление результата. Скриншот ниже снят с этого представления.

```
SELECT p.id, p.name, p.article, p.price,
       c.name AS category, p.created_at
FROM parts p
JOIN categories c ON c.id = p.category_id
ORDER BY p.id;
```

id	name	article	price, ₽	category	created_at (UTC)
1	Масляный фильтр	AP-FL-001	490	Двигатель	2026-10-30 18:57:36.131148
2	Тормозной диск	AP-BR-002	3200	Тормоза	2026-10-30 18:57:36.131154
3	Свеча зажигания	AP-IG-003	650	Зажигание	2026-10-30 18:57:36.131155
4	Воздушный фильтр	AP-FL-004	890	Двигатель	2026-10-30 18:57:36.131155
5	Амортизатор	AP-SU-005	4500	Подвеска	2026-10-30 18:57:36.131156
6	Поликлиновой ремень	AP-DR-006	1100	Привод	2026-10-30 18:57:36.131156
8	Салонный фильтр	AP-CB-007	790.5	Двигатель	2026-10-30 18:57:39.709989

Рисунок 3 — Фактические строки таблицы parts после добавления записи через API

Запись AP-CB-007 создана HTTP-запросом POST и присутствует в физическом файле БД. Полный снимок с SQL, источником и DDL таблиц: docs/screenshots/03-database.png. Запрос и полученный JSON-ответ: docs/http-demonstration.json.

Доказательства постоянного хранения

В интеграционном тесте первый процесс Python создаёт запись с ценой 123.45, завершает работу, затем второй процесс открывает тот же файл и читает цену. Это проверяет сохранение вне памяти процесса.

В браузерном сценарии после добавления записи процесс Uvicorn останавливается и запускается заново. При обновлении страницы запись остаётся. Изменённая цена и категория также сохраняются; удалённая запись не появляется после нового чтения.

Непосредственная проверка ограничений

Через прямые SQL-запросы проверено отклонение отрицательной цены, дробных копеек, NULL в обязательном поле, повторного артикула, отсутствующей категории и удаления используемой категории. Это подтверждает ограничения на уровне БД независимо от клиентской формы и Pydantic.

7. Результаты проверки и вывод

Проверка	Результат
Список, создание, чтение по ID, изменение, удаление	200 / 201 / 200 / 200 / 204
Чтение отсутствующего или удалённого ID	404
Повторный артикул; откат транзакции	400; следующий корректный запрос проходит
Обязательные поля, типы, длины, диапазоны	Ошибки 422, недопустимые значения не записаны
Пустой PATCH; повреждённый JSON	400
NOT NULL, UNIQUE, CHECK, FOREIGN KEY	Нарушения отклоняются SQLite
Перезапуск двух процессов Python	Запись сохраняется
Удаление всех деталей и повторная инициализация	Каталог остаётся пустым; данные не восстанавливаются
Swagger UI и спецификация OpenAPI	Методы доступны; запрос из Swagger возвращает 200
CRUD через браузер и повторная загрузка	Добавление, редактирование и удаление работают
Ошибка сети и повторный запрос	Понятное сообщение; соединение восстанавливается
Экран 390 px	Горизонтальной прокрутки нет
Раздельный frontend :5500 и backend :8000	Загрузка через CORS работает
Ошибки JavaScript	Не обнаружены

API: 44 интеграционных теста пройдены. Браузерные сценарии и сетевые методы проверены в Chromium. Протоколы: docs/api-tests.txt и docs/browser-tests.txt. Проверенная платформа — Linux, Python 3.12; отдельный запуск start.bat на Windows не проводился.

Вывод

Существующий статический каталог преобразован в клиент-серверное приложение. Реализованы связанные сущности, реляционная схема, полный REST CRUD, проверка данных, автоматическая документация и интерфейс, работающий через fetch. Данные сохраняются в SQLite после закрытия браузера и перезапуска backend.

В составе работы представлены исходный код, файл демонстрационной БД, README с установкой и примерами запросов, скриншоты, тесты и краткая инструкция для защиты. Источником постоянных данных служит БД; объект state в браузере используется только для отображения загруженного ответа.